

PipeSense-ARM: A Reproducible RTL Artifact for Safe Adaptive Pipeline Reconfiguration in an ARM-Like Embedded Core

tyboy
Independent researcher
United States
tyboy@users.noreply.github.com

Abstract- This paper presents PipeSense-ARM, a SystemVerilog research artifact for safe adaptive pipeline reconfiguration in a small ARM-like educational core. The prototype combines a five-stage pipeline, a narrow hardware observer, a mode controller with mode-specific stability and residency thresholds, and a drain-before-switch reconfiguration unit. Across 13 embedded-style workloads and six execution configurations, adaptive PipeSense-ARM reduces total cycles by 7.59% and total activity-energy proxy by 5.78% relative to static normal mode, improves 9 workloads, ties 3, and slows 1. A stricter oracle comparison shows an average -8.90% cycle gap to the best fixed mode, so the result is useful but not final. All directed HDL rows report zero safety faults and match a sequential ISA reference model. A 500-seed constrained-random safety run reports zero assertion failures, zero safety faults, and zero timeouts. Yosys reports both standalone and integrated generic-cell area proxies; these are not calibrated synthesis results. The contribution is a reproducible undergraduate architecture artifact, not an ARM-compatible processor or commercial optimization.

Index Terms- computer architecture, embedded processors, adaptive microarchitecture, pipeline reconfiguration, hardware performance monitoring

INTRODUCTION

Small embedded cores often run phase-changing code: control loops branch, signal-processing kernels stream through memory, dependency-heavy code stresses bypass paths, and idle regions reward lower activity. A static pipeline policy is simple, but it can be mismatched to the current bottleneck. PipeSense-ARM asks a scoped artifact question: can a small in-core observer classify pipeline behavior and safely select among visible microarchitectural modes at runtime?

PipeSense-ARM implements a simplified ARM/RISC-style five-stage IF/ID/EX/MEM/WB core with valid bits, stalls, flushes, forwarding, load-use hazard handling, memory waits, writeback, and performance counters. Around it, the artifact adds a pipeline observer, adaptive controller, and reconfiguration unit. The design is intentionally inspectable: the observer sees pipeline events rather than benchmark labels or software hints, and every claimed number is generated by scripts in the repository.

The contribution is not that branch optimization, memory buffering, hazard bypassing, or low-power policy are individually new. Prior work studies branch prediction [1981], prefetch and victim buffering [1990], program phases [2002], dynamic hardware configuration [2002, 2000], and architectural power modeling [2000, 2001, 2003]. The contribution here is the integrated, reviewable control loop and its evidence trail: static baselines, an oracle comparison, validation against a sequential reference model, safety

fuzzing, sweep data, and generic-cell Yosys cost.

Three artifact properties drive the design. First, every mode changes an observable execution path, so the controller is not only relabeling phases. Second, every result row includes safety and architectural-consistency fields, so a faster run cannot be used if it corrupts the modeled machine state. Third, the paper checker reads generated CSVs and rejects stale manuscript tables. These constraints make the current claims narrower, but they also make the artifact easier for a reviewer to reproduce and challenge.

BACKGROUND AND RELATED WORK

Architecture evaluation separates mechanism, policy, and workload. PipeSense-ARM exposes stylized mechanisms as modes, then asks whether a lightweight hardware policy can choose among them from local event windows. `MODE_BRANCH_OPT` models lower branch penalty, `MODE_MEMORY_OPT` models deterministic memory-wait mitigation, `MODE_HAZARD_OPT` models a stronger load-result bypass path, and `MODE_LOW_POWER` lowers an activity proxy. These are not full branch predictors, caches, prefetchers, power gates, or product-ready mechanisms.

The measurement framing follows executable-modeling and quantitative-methodology traditions [2002, 2017]. The safety framing uses ideas from SystemVerilog Assertions, assertion-based design, model checking, and reconfigurable-system verification [2023, 2004, 1986, 2007]. The artifact provides simulation monitors, randomized safety regression, and bounded abstract formal scaffolding; it is not formal verification of the full processor.

This positioning is deliberately conservative. A stronger architecture paper would replace the stylized modes with realistic branch, memory, and clocking structures, then calibrate timing and energy on a named target. PipeSense-ARM instead focuses on the smaller question of whether a hardware-native control loop can be implemented, measured, and checked without relying on hidden software labels or hand-edited result tables.

PIPESENSE-ARM ARCHITECTURE

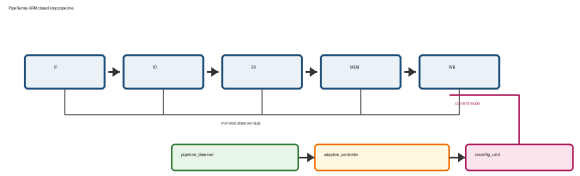


Fig. 1. PipeSense-ARM places an observer/controller/reconfiguration loop around a small five-stage pipeline.

Fig. the corresponding table or figure shows the closed-loop structure. The core supports ADD, SUB, AND, ORR, EOR, LDR, STR, B, CMP, NOP, and a simulation-only HALT sentinel. The observer samples stage-valid bits, IF/ID/EX stalls, branch flushes and taken events, load-use hazards, memory waits, retired instructions, and cycle count. Over a rolling window it classifies behavior as balanced, branch-heavy, memory-stall, load-use-hazard, front-end-stall, or idle/low-utilization.

The controller maps phases to modes with mode-specific stability and residency thresholds. Branch mode requires a

stable branch-heavy signal; memory mode can enter quickly because memory stalls are costly; hazard mode uses a shorter stability requirement; low-power and normal modes require longer residency to avoid thrashing. Front-end stalls prefer branch mode unless the current mode is already memory-optimized. Balanced windows are sticky: the controller keeps the current mode rather than forcing an immediate return to normal.

The v2 controller is a direct response to the first draft's oracle gaps. The earlier policy used one residency threshold for all transitions, which delayed memory-mode entry and wasted short useful phases. The current controller keeps the observer thresholds simple but adds mode-specific transition requirements. This improves the memory-heavy and generated-control cases without hiding the remaining negative dsp-fir-codegen row.

Mode request and mode commit are separated. A requested mode can be pending while the reconfiguration unit waits for a safe boundary. Logs therefore distinguish requested mode, current mode, committed reconfigurations, and reconfiguration penalty cycles. This distinction matters because short phases can be recognized too late to justify a drain.

The hardware paths are intentionally small enough to inspect in one repository. `arm_like_core.sv` integrates the pipeline and mode behavior. `pipeline_observer.sv` implements rolling event counters and phase classification. `adaptive_controller.sv` implements hysteresis and mode requests. `reconfig_unit.sv` handles fetch gating, drain, mode commit, and penalty accounting. Separate scripts generate the same benchmark programs for the HDL and sequential reference model so parity can be checked automatically.

SAFE RECONFIGURATION AND IMPLEMENTATION

The reconfiguration unit gates fetch when a new mode is requested, then waits until IF/ID, ID/EX, EX/MEM, and MEM/WB are empty and no memory wait is active. Only then does it install the requested mode and increment reconfiguration counters. This conservative protocol prevents mode changes from altering branch, memory, or forwarding behavior while older instructions are still in flight.

The testbench tags instructions as they move through the pipeline. Monitors flag illegal mode changes outside empty-pipeline boundaries, completion before drain, fetch not gated during active reconfiguration, duplicate or nonmonotonic retirement tags, and reference-model mismatches. A result row is usable only when `safety_faults == 0`, `timed_out == 0`, and the HDL retired count and architectural data-state hash match the sequential ISA model.

The repository also includes bindable safety assertions, functional coverage counters, and abstract formal harnesses for reconfiguration and token conservation. The 500-seed fuzz regression produces 2,500 mode-result rows with zero assertion failures, zero safety faults, and zero timeouts. The formal files are useful scaffolding, but they are not formal verification of the full RTL core.

The safety policy is intentionally conservative because the modes affect in-flight behavior. A branch-mode switch can change flush timing, a hazard-mode switch can change bypass assumptions, and a memory-mode switch can change wait behavior around outstanding memory operations. Waiting for an empty pipeline is slower than selective quiescence, but it gives the artifact a simple invariant: no instruction retires under a different mode than the one governing its already-issued pipeline behavior.

EVALUATION METHODOLOGY

The evaluation uses 13 embedded-style workloads: arithmetic-heavy, branch-heavy, memory-heavy, load-use-heavy, mixed-control, tiny-FIR, dhrystone-toy, coremark-toy, dsp-fir-codegen, pid-control-codegen, long-FIR-stress, PID-phase-stress, and randomized-memory-latency-stress. The last three are longer stress workloads added to reduce overfitting to tiny phase-biased programs. Each workload runs in static normal, fixed branch, fixed memory, fixed hazard, fixed low-power, and adaptive modes.

The workloads are intentionally transparent rather than benchmark-suite complete. Arithmetic-heavy isolates ALU forwarding and writeback timing. Branch-heavy stresses CMP-to-branch behavior and flush accounting. Memory-heavy and randomized-memory-latency-stress exercise wait mitigation under deterministic and irregular address patterns. Load-use-heavy and long-FIR-stress stress bypass and load-result timing. Mixed-control combines branches, loads, stores, and dependencies. Tiny-FIR, dsp-fir-codegen, pid-control-codegen, and PID-phase-stress are small embedded-kernel analogues that make the workload set less anonymous while preserving instruction-for-instruction comparability with the reference model.

Metrics include cycles, retired instructions, IPC, stalls, flushes, memory waits, load-use stalls, reconfiguration count, reconfiguration penalty, activity-energy proxy, safety faults, timeout status, and final architectural hash. The activity-energy value is an RTL proxy, not physical power. The Yosys data is a generic-cell area proxy and not calibrated synthesis results, FPGA utilization, timing closure, or silicon area.

The evaluation order is fixed. Simulation output is parsed into raw result CSVs, the validator checks modes, benchmark coverage, timeouts, and safety faults, the reference model checks retired counts and architectural hashes, and only then do comparison scripts compute static-normal improvement and oracle gap. The paper checker runs last so that changed RTL, workloads, or synthesis proxy results cannot leave stale tables in the manuscript.

This ordering is also used when packaging the artifact. The full local run produces raw HDL logs, normalized CSVs, reference-model CSVs, disassembly, sweep directories, fuzz summaries, area reports, generated figures, and a rendered paper preview. The checked-in summary gives reviewers a quick status page, while the detailed artifacts remain reproducible by rerunning the scripts.

RESULTS

The current directed HDL run contains 78 benchmark/mode rows. Every run halted, no run timed out, every run reported zero safety faults, and every adaptive row matched the sequential ISA reference model. Table the corresponding table or figure compares adaptive PipeSense-ARM against static normal mode.

Across all 13 workloads, static-normal execution takes 3,307 cycles and adaptive execution takes 3,056 cycles, a 7.59% total reduction. Total activity-energy proxy falls from 19,162 to 18,055, a 5.78% reduction. Adaptive improves cycles on 9 workloads, ties 3, and slows dsp-fir-codegen by 1.04%. The average per-workload cycle reduction is 6.33%, average IPC improvement is 7.71%, and average activity-energy reduction is 5.12%.

Table the corresponding table or figure compares adaptive mode with the best fixed mode chosen after seeing each workload. Negative gaps mean adaptive is worse than that hindsight oracle.

The oracle comparison keeps the claim bounded. Adaptive remains an average -8.90% behind the best fixed mode in cycles and -5.43% behind in activity-energy proxy. This shows that the mode mechanisms have useful headroom, while the online controller still misses fixed-memory opportunities on mixed-control and tiny-FIR.

The most important change from the earlier draft is that adaptive memory behavior is now plausible but still imperfect. Memory-heavy improves by 29.87% versus static normal and closes most of the previous fixed-memory gap, but it remains 10.20% behind the oracle. Mixed-control and tiny-FIR still show larger oracle gaps because the online policy sees mixed evidence and pays reconfiguration cost. The single static-normal regression, dsp-fir-codegen, is kept in the table because it is exactly the kind of short generated loop where controller overhead can dominate.

The ablation rows make the controller's contribution visible. Disabling the observer or controller collapses adaptive execution back to the static-normal aggregate. The zero-cost row is not an unsafe RTL run; it subtracts measured reconfiguration penalty from validated adaptive rows to estimate a best-case bound for reducing drain cost. That bound is small after controller v2, which suggests that the next improvement should target phase decisions rather than only switch latency.

The area proxy is intentionally sobering. Standalone observer, controller, and reconfiguration modules sum to 2,885 generic cells, and the integrated proxy reports 4,850 cells, a 165.03% delta over the baseline core proxy. These numbers should not be read as final silicon cost, but they do warn that the first prototype spends more hardware on observation than a tiny teaching core can justify without better mechanisms or shared counters.

The row-level CSVs are kept because aggregate wins alone would overstate the artifact. For example, adaptive memory behavior is strong on memory-heavy but weaker on mixed-control, where the best fixed mode still wins by 20.95%. Likewise, the low cycle gap on long-FIR-stress does not imply general robustness; it means that this longer workload happens to align with the current memory-entry policy. These details are easier to audit in CSV form than in prose, so the repository keeps both summary tables and raw generated outputs.

Table I. Adaptive PipeSense versus static-normal baseline.

Benchmark	Norm	Adapt	Cycle	IPC	Energy	Rcfg
arithmetic heavy	30	30	0.00%	0.00%	0.00%	0
branch heavy	128	114	10.94%	12.28%	6.64%	1
coremark toy	162	162	0.00%	0.00%	0.00%	0
dhystone toy	133	133	0.00%	0.00%	0.00%	0
dsp fir codegen	192	194	-1.04%	-1.03%	0.36%	2
load use heavy	205	178	13.17%	15.17%	7.64%	1
long fir stress	876	792	9.59%	10.62%	7.63%	1
memory heavy	231	162	29.87%	42.60%	26.81%	1
mixed control	131	127	3.05%	3.14%	2.28%	1
pid control codegen	192	188	2.08%	2.13%	1.49%	1
pid phase stress	653	620	5.05%	5.32%	2.62%	1
random mem latency stress	215	204	5.12%	5.39%	5.05%	1
tiny fir	159	152	4.40%	4.61%	6.00%	2

Table II. Adaptive mode versus best fixed-mode oracle.

Benchmark	Best fixed	Best	Adapt	Cycle gap	Energy gap
arithmetic heavy	hazard	29	30	-3.45%	-1.73%
branch heavy	branch	105	114	-8.57%	-3.94%
coremark toy	hazard	150	162	-8.00%	-3.24%
dhystone toy	branch	124	133	-7.26%	-3.38%
dsp fir codegen	hazard	168	194	-15.48%	-5.35%
load use heavy	hazard	169	178	-5.33%	-2.16%
long fir stress	memory	778	792	-1.80%	-1.14%
memory heavy	memory	147	162	-10.20%	-7.39%
mixed control	memory	105	127	-20.95%	-18.31%
pid control codegen	memory	178	188	-5.62%	-4.35%
pid phase stress	memory	611	620	-1.47%	-2.28%
random mem latency stress	memory	189	204	-7.94%	-5.05%
tiny fir	memory	127	152	-19.69%	-12.31%

Table III. Yosys generic-cell area proxy, not calibrated synthesis.

Component	Cells	Overhead
baseline core proxy	1830	100.00%
pipeline_observer	2128	116.28%
adaptive_controller	182	9.95%
reconfig_unit	575	31.42%
integrated core proxy	4850	165.03%

Component	Cells	Overhead
observer/controller/reconfig sum	2885	157.65%

PARAMETER SENSITIVITY

The repository runs a full 3x3x3 sweep over observer window size, minimum mode residency, and threshold profile. The window settings are 16, 32, and 64 cycles; the residency settings are 8, 24, and 48 cycles; and the threshold profiles are tight, medium, and loose. Each setting runs all 78 benchmark/mode rows and is validated with the same CSV checks. The sweep produces 2,106 result rows plus adaptive-versus-fixed comparison files. It is part of the artifact because the controller is policy-sensitive: good results at one threshold point are not enough to claim a robust processor technique. CI now runs the full sweep rather than a smoke subset.

The sweep is not used to cherry-pick a new default in this paper. Instead, it is a stress check for the control policy and a map for future tuning. If a reviewer changes thresholds or residency and reruns the scripts, the validator, reference comparison, oracle comparison, and safety checks still have to pass. That makes parameter exploration part of the artifact rather than an informal notebook exercise.

DISCUSSION AND LIMITATIONS

PipeSense-ARM supports a modest claim. A small hardware-resident observer can guide safe mode switching in a sequential pipeline prototype and improve over static normal mode on several workloads. It does not beat the oracle fixed-mode baseline, and the largest remaining misses show controller headroom rather than a finished optimization.

The main threats to validity are workload representativeness, stylized modes, deterministic memory behavior, proxy energy, and limited safety scope. The workloads are more recognizable than the first draft, but they are still toy-ISA kernels rather than full compiler-generated embedded suites. The Yosys results are generic-cell proxies, not calibrated FPGA, timing, power, or physical-area evidence. The safety evidence combines monitors, assertions, fuzzing, and abstract formal scaffolding, but it is not formal verification of the full RTL processor.

The most direct next step is to turn each stylized mode into a real mechanism: a small branch predictor or early-branch unit, a stream buffer or scratchpad policy, a concrete load-result bypass path, and a target-specific low-power model. The same artifact checks should remain in place after those changes. In particular, any stronger performance claim should still include static normal, all fixed modes, oracle gap, sweep sensitivity, fuzz safety, reference comparison, and calibrated hardware cost.

Another limitation is that the controller is still threshold-based. That is useful for a first artifact because the counters and comparators are inspectable, but it cannot reason about phase duration, upcoming loop exits, or interaction between memory and branch events. The oracle table shows exactly where such context would help. A future policy could retain the same safe reconfiguration protocol while replacing the threshold mapper with a small finite-state predictor, a table-driven controller, or offline-tuned thresholds derived from longer benchmark traces.

ARTIFACT AND REPRODUCIBILITY

The artifact is organized so paper claims can be checked. `scripts/run_sim.py` compiles and runs the HDL matrix, validates CSVs, runs the ISA reference model, and compares retired counts and architectural hashes. `scripts/run_sweep.py`, `verif/fuzz_runner.py --seeds 500`, `scripts/synth_area_report.py`, and `scripts/check_paper.py` cover sweep, safety, area-proxy, and manuscript consistency. The GitHub workflow installs Icarus Verilog and Yosys, then runs simulation, validation, reference comparison, full sweep, 500-seed fuzzing, Yosys area proxy, paper checking, and preview generation.

The CI path is intentionally heavier than a smoke test because the paper is an artifact paper. It reruns the checks reviewers are likely to care about: no hidden timeouts, no nonzero safety faults, no missing benchmark/mode combinations, no stale paper numbers, no omitted oracle comparison, and no missing Yosys status. If a dependency cannot run, the expected behavior is to record the blocker rather than invent output.

Generated CSVs, logs, build products, and preview files are reproducible artifacts. `results/SUMMARY.md` records which scripts ran cleanly and the headline numbers visible from GitHub. The manuscript source remains a five-page URTC-targeted workshop draft; final submission still requires the user's preferred author metadata and any venue-specific formatting checks.

The manuscript checker intentionally treats the paper as another generated artifact consumer. It verifies citation keys against the bibliography, rejects unresolved placeholder tokens, checks claim-discipline phrases, and compares every adaptive, oracle, ablation, and area table row against the current CSVs. This prevents the common failure mode where code improves, generated data changes, and the paper silently keeps last week's numbers.

CONCLUSION

PipeSense-ARM is best framed as a reproducible RTL artifact for hardware-native adaptive processor-control experiments. It includes a real pipeline model, event observer, controller v2 with mode-specific thresholds, safe drain-before-switch reconfiguration, thirteen workloads, reference-model validation, fuzz safety evidence, sweep data, and integrated Yosys cost. The results improve over static normal mode while staying behind the oracle fixed-mode baseline, which gives reviewers both a concrete contribution and a clear boundary.

REFERENCES

- [1] J. E. Smith, A Study of Branch Prediction Strategies, ISCA, 1981.
- [2] N. P. Jouppi, Improving Direct-Mapped Cache Performance by the Addition of a Small Fully-Associative Cache and Prefetch Buffers, ISCA, 1990.
- [3] T. Sherwood, E. Perelman, G. Hamerly, and B. Calder, Automatically Characterizing Large Scale Program Behavior, ASPLOS, 2002.
- [4] A. S. Dhodapkar and J. E. Smith, Managing Multi-Configuration Hardware via Dynamic Working Set Analysis, ISCA, 2002.

- [5] R. Balasubramanian, D. H. Albonesi, A. Buyuktosunoglu, and S. Dwarkadas, Memory Hierarchy Reconfiguration for Energy and Performance, MICRO, 2000.
- [6] D. Brooks, V. Tiwari, and M. Martonosi, Wattch: A Framework for Architectural-Level Power Analysis and Optimizations, ISCA, 2000.
- [7] T. Mudge, Power: A First-Class Architectural Design Constraint, Computer, 2001.
- [8] C. Isci and M. Martonosi, Runtime Power Monitoring in High-End Processors, MICRO, 2003.
- [9] T. Austin, E. Larson, and D. Ernst, SimpleScalar: An Infrastructure for Computer System Modeling, Computer, 2002.
- [10] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed., 2017.
- [11] IEEE, IEEE Standard for SystemVerilog--Unified Hardware Design, Specification, and Verification Language, IEEE Std 1800-2023, 2023.
- [12] H. D. Foster, A. C. Krolnik, and D. J. Lacey, Assertion-Based Design, Kluwer Academic Publishers, 2004.
- [13] E. M. Clarke, E. A. Emerson, and A. P. Sistla, Automatic Verification of Finite-State Concurrent Systems Using Temporal Logic Specifications, TOPLAS, 1986.
- [14] M. Borgatti et al., An Integrated Design and Verification Methodology for Reconfigurable Multimedia Systems, arXiv:0710.4846, 2007.